

30-Day FreeRTOS Course for ESP32 Using ESP-IDF

(Day 14)

**“Hands-on Mini Project:
UART Logger with Mutex”**



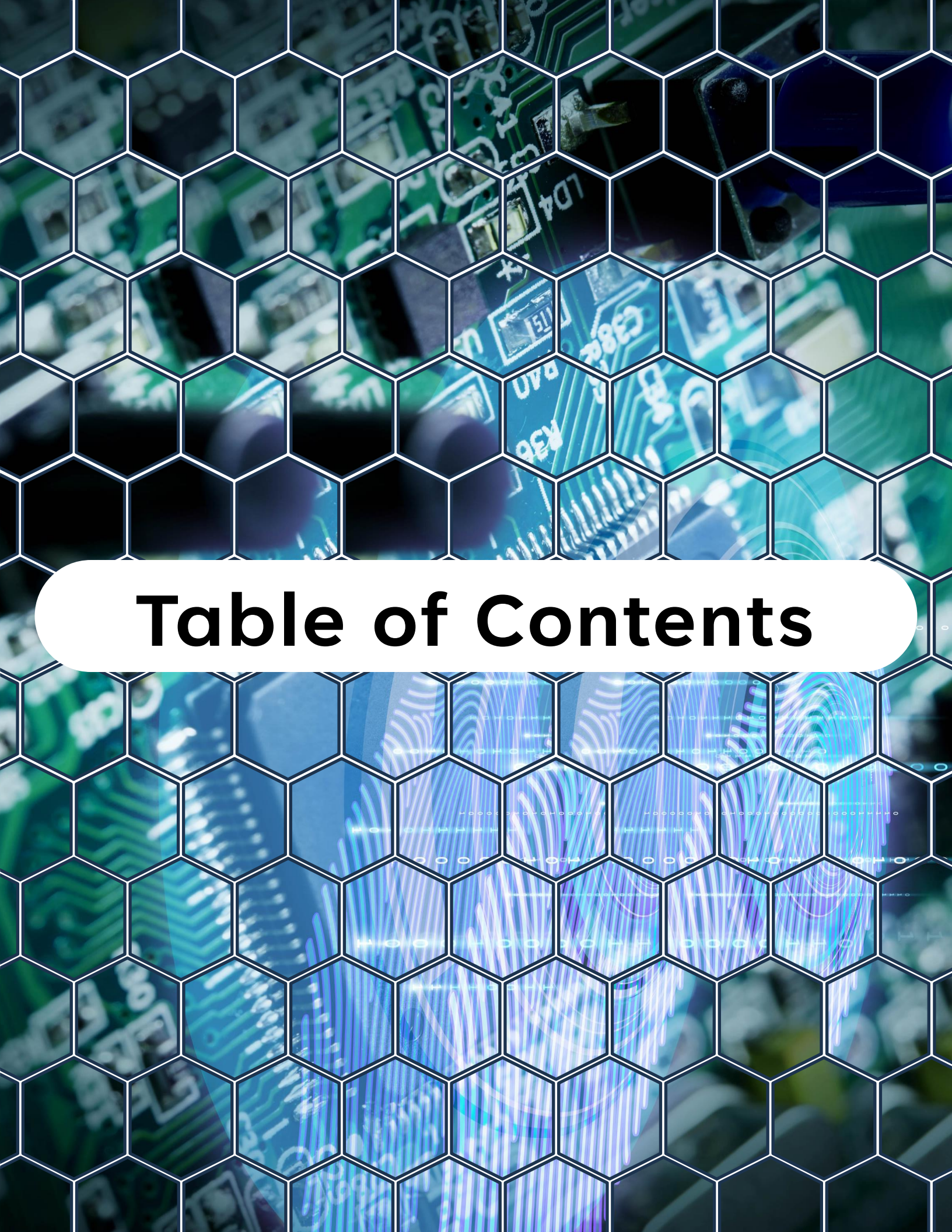
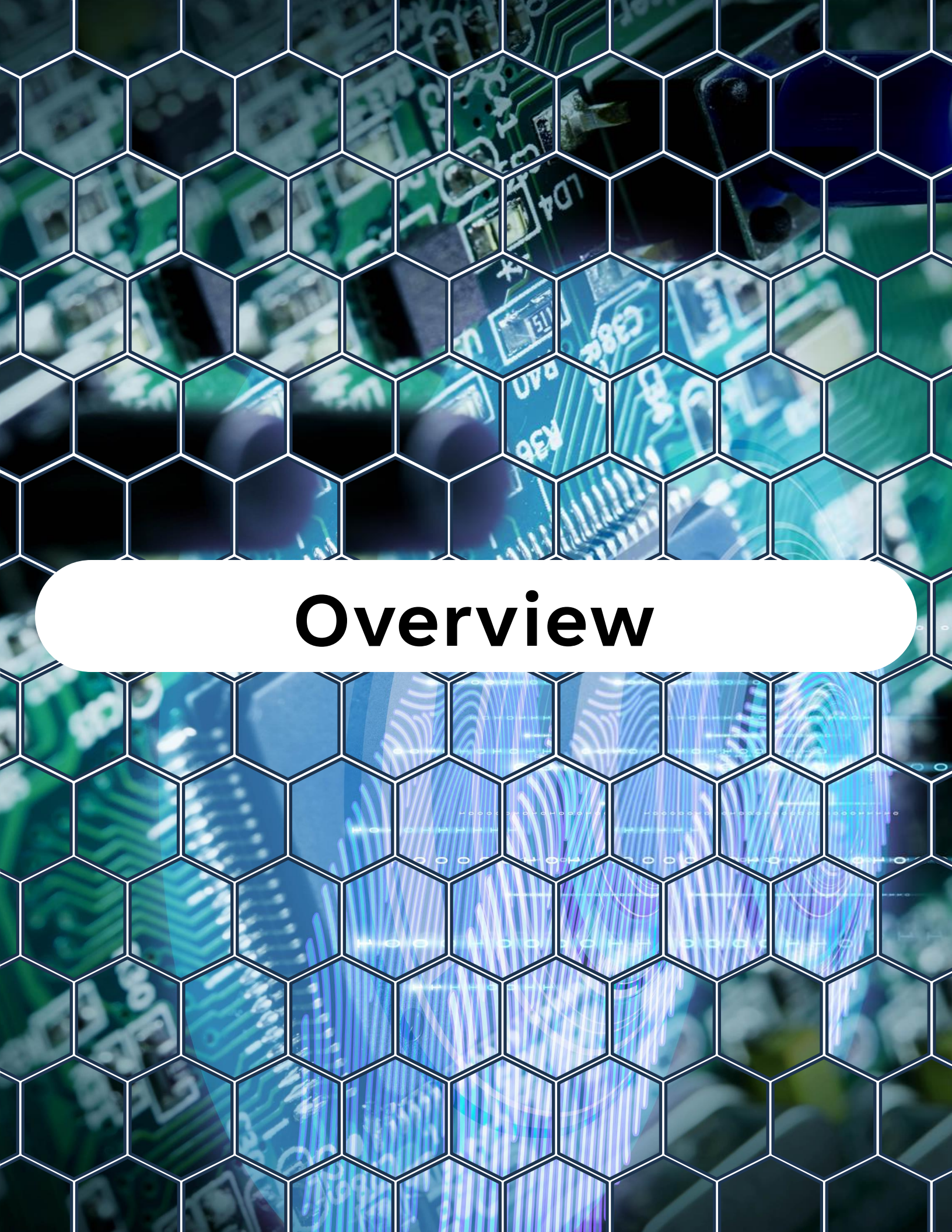


Table of Contents

Table of Contents

1. Overview
2. Why Use a Mutex for UART Logging?
3. System Design
4. Example – UART Logger with Mutex
5. Best Practices
6. Summary
7. Challenge for Today



Overview

Overview

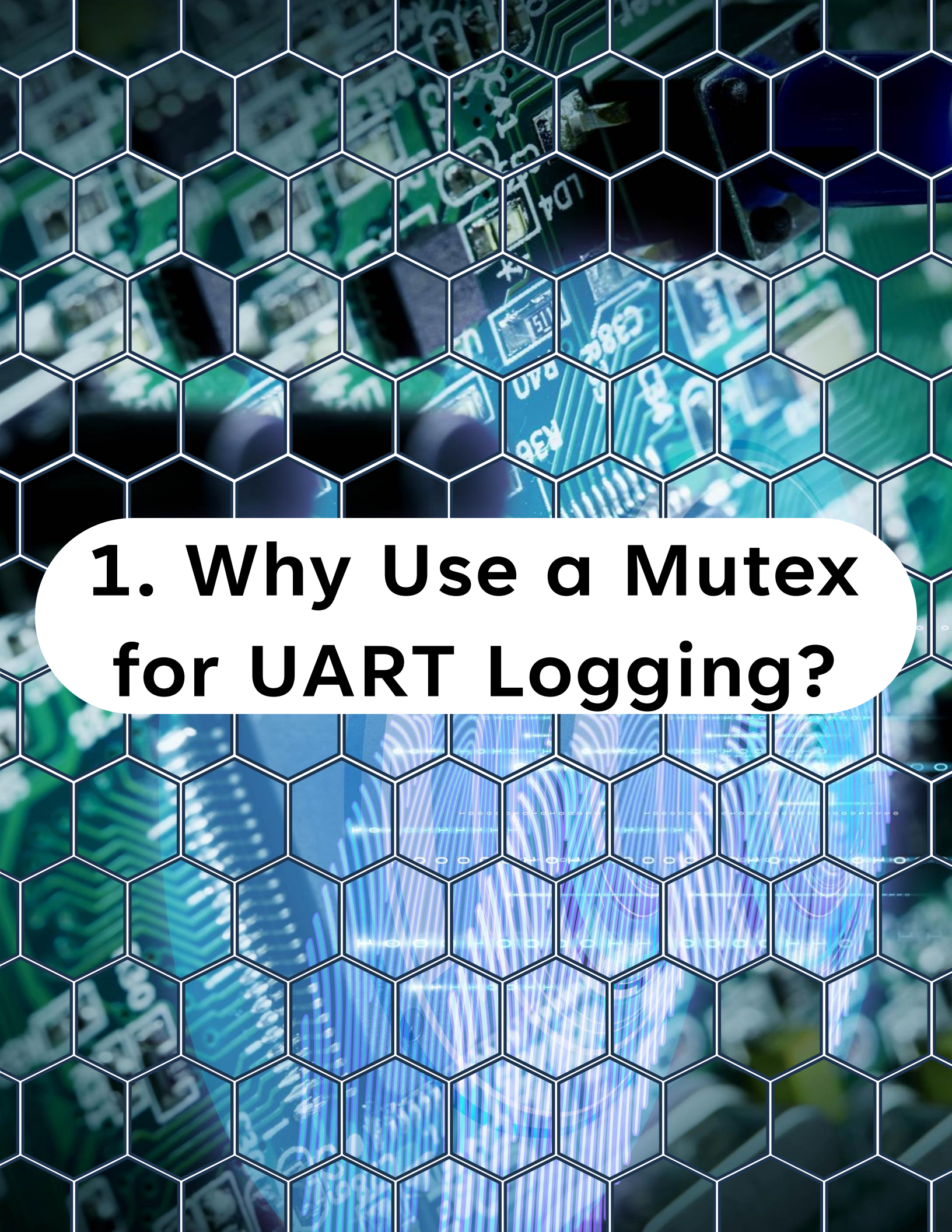
Today you'll build a **thread-safe UART logger** using a FreeRTOS mutex.

Multiple tasks will try to write to the UART simultaneously, and the mutex will ensure clean, synchronized output.

You'll learn:

- How to **configure UART** in ESP-IDF
- How to create a **mutex-protected logging system**
- How multiple tasks can safely share a common resource
- The difference between **protected vs unprotected logging**

By the end of this lesson, you'll have a working **UART logger service** for FreeRTOS applications on ESP32.



1. Why Use a Mutex for UART Logging?

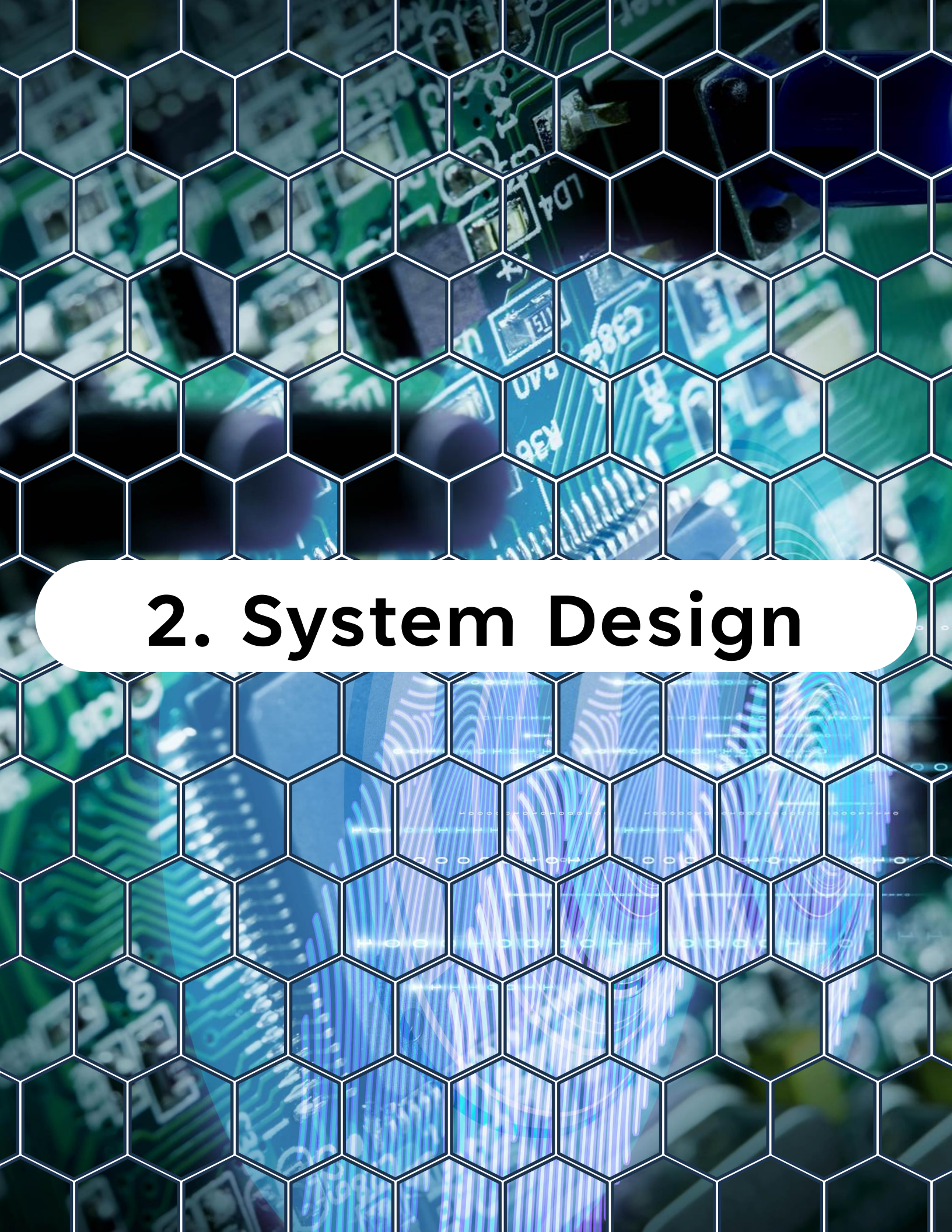
1. Why Use a Mutex for UART Logging?

UART is a **shared peripheral**. If multiple tasks write to it without coordination:

 Messages interleave → output becomes corrupted.

 With a mutex, tasks take turns writing, ensuring logs stay readable.

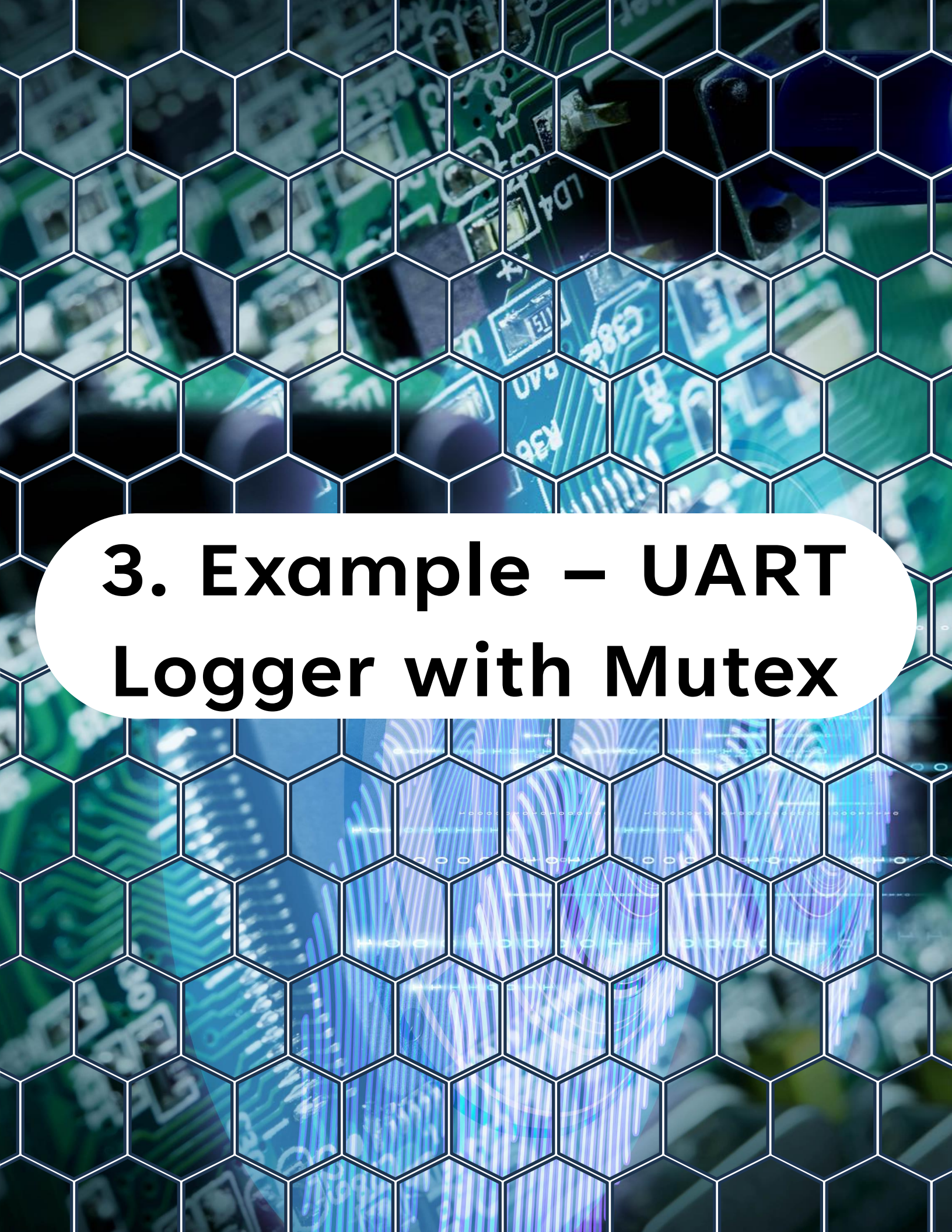
This project demonstrates how to **safely share UART between tasks**.



2. System Design

2. System Design

- **UART Logger Mutex** protects access to the UART peripheral.
- **Task A** writes messages every 500 ms.
- **Task B** writes messages every 800 ms.
- Both tasks acquire the mutex before writing, then release it afterward.



3. Example – UART Logger with Mutex

3. Example – UART Logger with Mutex

Code Example



```
1  #include <stdio.h>
2  #include <string.h>
3  #include "freertos/FreeRTOS.h"
4  #include "freertos/task.h"
5  #include "freertos/semphr.h"
6  #include "driver/uart.h"
7
8  #define TAG "DAY14"
9  #define UART_PORT UART_NUM_0
10 #define BUF_SIZE 1024
11
12 SemaphoreHandle_t uart_mutex;
13
14 // Function to log messages safely from multiple tasks
15 void uart_logger(const char *msg) {
16     // Take the mutex before accessing UART
17     if (xSemaphoreTake(uart_mutex, portMAX_DELAY)) {
18         uart_write_bytes(UART_PORT, msg, strlen(msg));
19         uart_write_bytes(UART_PORT, "\n", 1);
20         // Release the mutex
21         xSemaphoreGive(uart_mutex);
22     }
23 }
24
25 // Example tasks that log messages
26 void taskA(void *pvParameter) {
27     while (1) {
28         uart_logger("Task A: Writing log message...");
29         vTaskDelay(pdMS_TO_TICKS(500));
30     }
31 }
```

3. Example – UART Logger with Mutex

```
32
33 // Another example task that log messages
34 void taskB(void *pvParameter) {
35     while (1) {
36         uart_logger("Task B: Writing log message...");
37         vTaskDelay(pdMS_TO_TICKS(800));
38     }
39 }
40
41 // Main application entry point
42 void app_main(void) {
43     // Create UART mutex
44     uart_mutex = xSemaphoreCreateMutex();
45     if (uart_mutex == NULL) {
46         printf("Failed to create mutex\n");
47         return;
48     }
49
50     // Configure UART
51     const uart_config_t uart_config = {
52         .baud_rate = 115200,
53         .data_bits = UART_DATA_8_BITS,
54         .parity = UART_PARITY_DISABLE,
55         .stop_bits = UART_STOP_BITS_1,
56         .flow_ctrl = UART_HW_FLOWCTRL_DISABLE
57     };
58     uart_driver_install(UART_PORT, BUF_SIZE, BUF_SIZE, 0, NULL, 0);
59     uart_param_config(UART_PORT, &uart_config);
60
61     // Create logging tasks
62     xTaskCreate(taskA, "TaskA", 2048, NULL, 5, NULL);
63     xTaskCreate(taskB, "TaskB", 2048, NULL, 5, NULL);
64 }
```


3. Example – UART Logger with Mutex

Expected Behavior:

With mutex:

Task A: Writing log message...

Task B: Writing log message...

Task A: Writing log message...

Task B: Writing log message...

Without mutex:

Task A: Writing log message...
Task B: Writing log message...
Task A: Writing log message...
Task B: Writing log message...

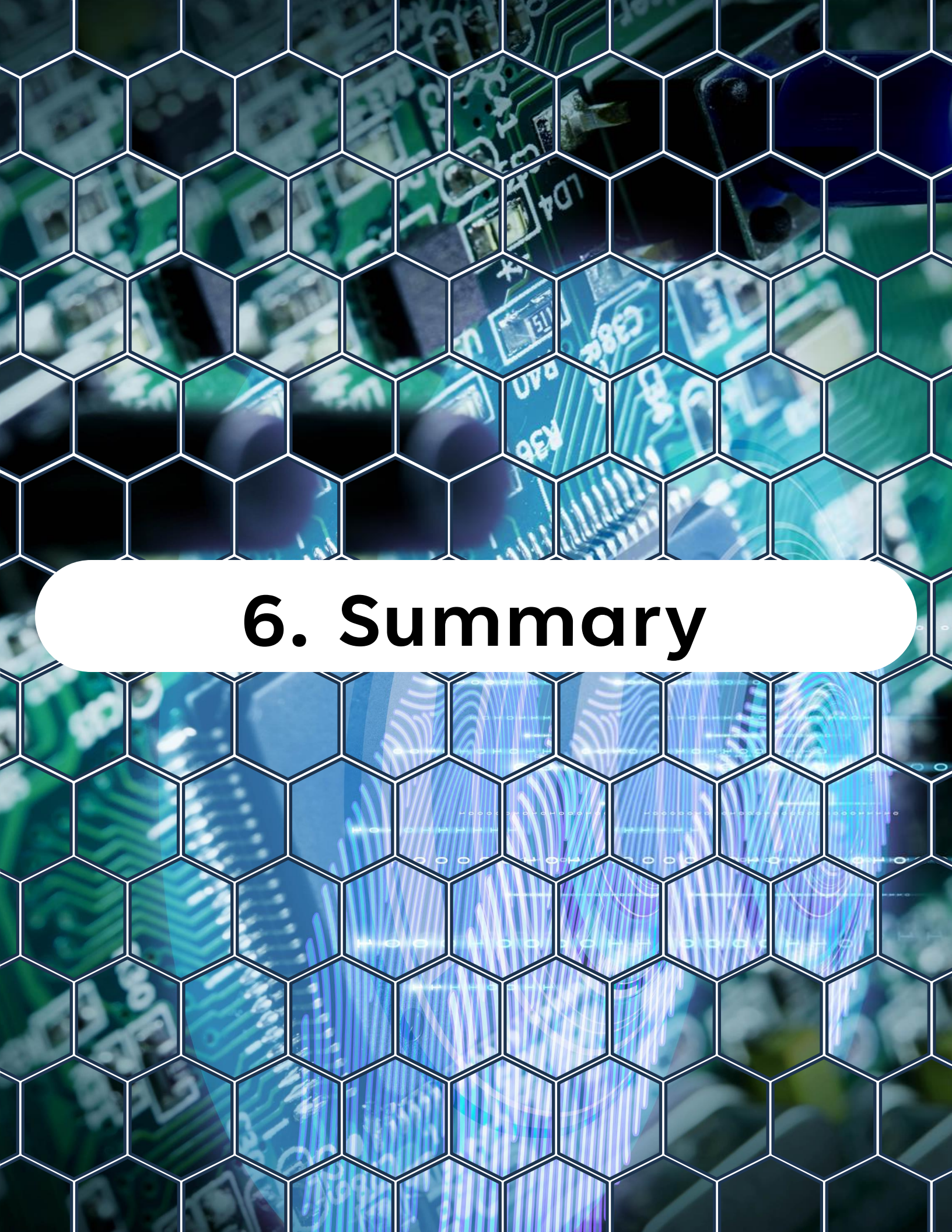
The mutex ensures **atomic UART writes**,
preventing garbled output.



5. Best Practices

5. Best Practices

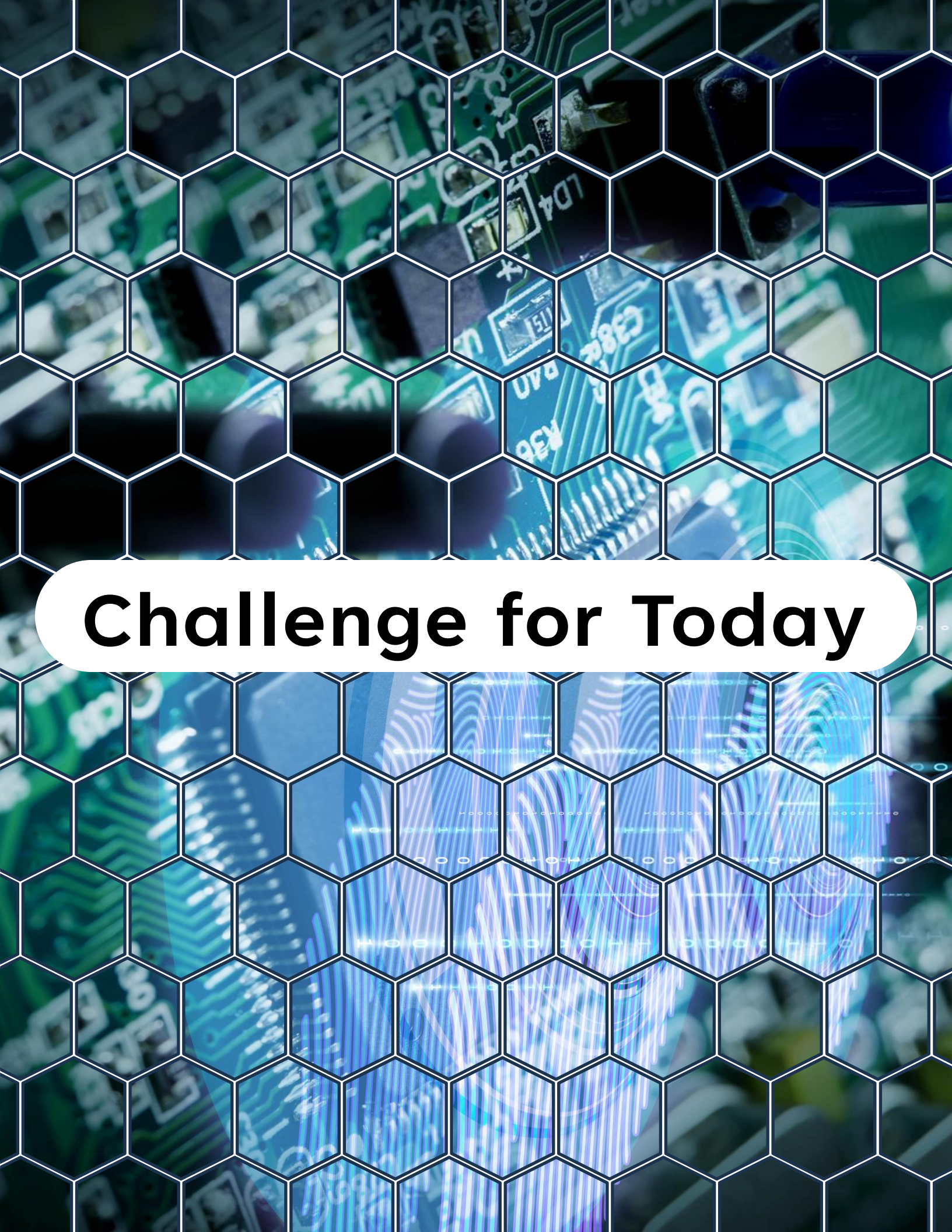
- ✓ Always use mutexes when multiple tasks share UART, SPI, or I2C.
- ✓ Keep critical sections short — don't hold the mutex longer than necessary.
- ✓ Never use mutexes inside ISRs — they are not ISR-safe.
- ✓ Consider using a dedicated logger task with a queue if logging is frequent (scales better).



6. Summary

6. Summary

- Mutexes provide safe, exclusive access to shared resources.
- UART logging tasks must use a mutex to prevent mixed output.
- ESP-IDF provides `uart_write_bytes()` for safe transmission.
- For heavy logging, design a **centralized logging service** with a queue.



Challenge for Today

Challenge for Today

Extend today's project by:

- Creating a **third task** that logs timestamps every 2 seconds.
- Compare logging with and without a mutex — observe the corruption.
- Bonus: Replace direct logging with a **queue-based logger task** (we'll cover queues in more detail later).

"Thanks for watching!

If you enjoyed this video,

*make sure to hit the like button and
subscribe to stay updated with my latest
content.*

*Don't forget to check out my other
videos for more tips and tutorials on
Embedded C, Python, hardware designs,
etc. Keep exploring, keep learning, and
I'll see you in the next video!"*



<https://www.linkedin.com/in/yamil-garcia>

<https://www.youtube.com/@LearningByTutorials>

<https://github.com/god233012yamil>